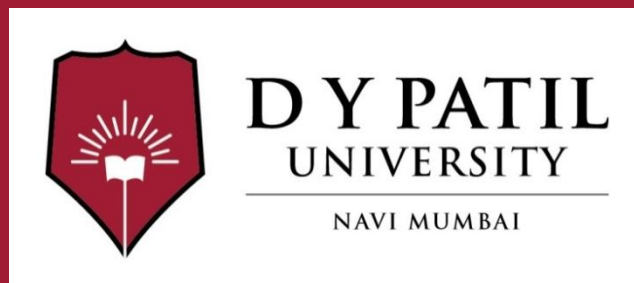


Subject Name :

Unit No: 3

Unit Name : Tree-Based Models and Ensemble Learning

Faculty Name :



Unit No.: 3.1

Faculty Name :

Decision Trees

- ✓ **Decision Trees (DT)** is a supervised learning technique. It can be used for both classification and regression. That is, they can either output a **categorical** prediction (*like if a plant is of a certain kind or not*) or a **numerical** prediction (*like the price of a house*).
- DT are **non parametric models**. This means that Decision trees are flexible models that don't increase their number of parameters as we add more features (if we build them correctly).
- **The goal of Decision Tree classifier is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.**
- ✓ A decision tree is a flowchart-like structure that uses a series of **"if-then"** statements (or "if-then-else") to model decisions and their possible outcomes.

Decision Trees

- A decision tree is a graphical representation of all the possible solution to a decision based on certain conditions.
- It works by recursively partitioning the data into subsets based on feature values, making decisions at each node to maximize a specific criterion (e.g., information gain or Gini index).
- It has a hierarchical, tree structure, which consists of a root node, branches, internal nodes and leaf nodes.
- A decision tree starts with a root node, which does not have any incoming branches.
- The outgoing branches from the root node then feed into the internal nodes, also known as decision nodes.
- Based on the available features, both node types conduct evaluations to form homogenous subsets, which are denoted by leaf nodes, or terminal nodes. The leaf nodes represent all the possible outcomes within the dataset.
- Decision trees can, unlike linear models, fit linearly inseparable datasets.

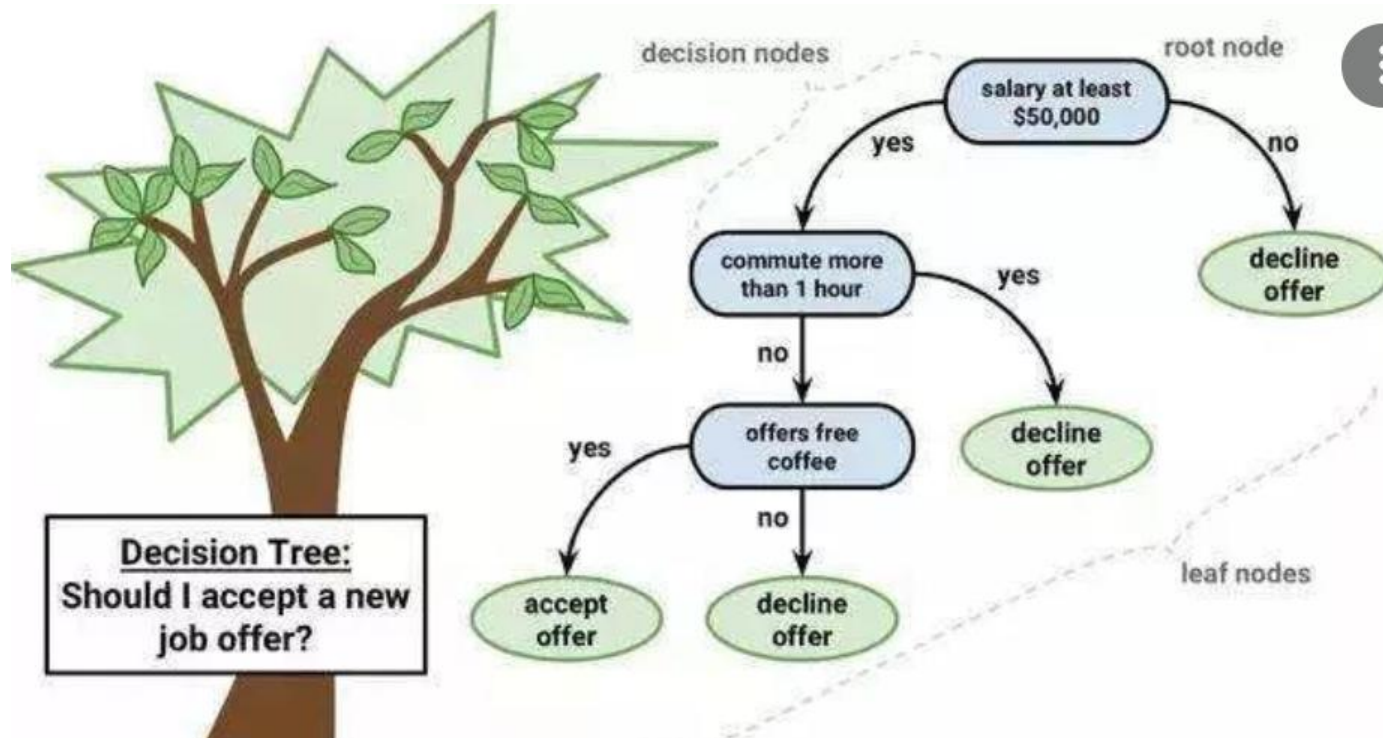
Key Components of Decision Tree

- ✓ • **Root Node:** The top node in the tree that represents the best feature to split the data.
- ✓ • **Internal Nodes:** Represent the features used for splitting the data based on specific decision rules.
- ✓ • **Leaf Nodes:** Terminal nodes that represent the predicted outcome (class label or numerical value).
- ✓ • **Branches:** Connections between nodes representing the possible values of the features.

Decision Tree

- ✓ They are constructed using two kinds of elements: **nodes and branches**. At each node, one of the features of our data is evaluated in order to split the observations in the training process or to make an specific data point follow a certain path when making a prediction.
- ★ A **decision tree** is a tree-like structure where each internal node represents a feature (or attribute), each branch denotes a decision rule, and each leaf node represents an outcome or prediction. It breaks down complex decisions into a series of simpler choices, making it easy to interpret and understand.

Should I except a new job offer or not ?



Working of Decision Tree Algorithm

1. Root Node Selection

- The process begins with the entire dataset.
- The best attribute (feature) to split on is selected based on a criterion like **Gini impurity**, **Entropy**, or **Mean Squared Error (MSE)** (for regression).

2. Splitting the Dataset

- The selected feature divides the dataset into subsets.
- The split is chosen to maximize information gain (i.e., to reduce impurity as much as possible).

3. Creating Branches & Nodes

- Each subset forms a branch leading to child nodes.
- The process continues recursively, selecting the best feature at each step to further split the data.

Working of Decision Tree Algorithm (cont'd)

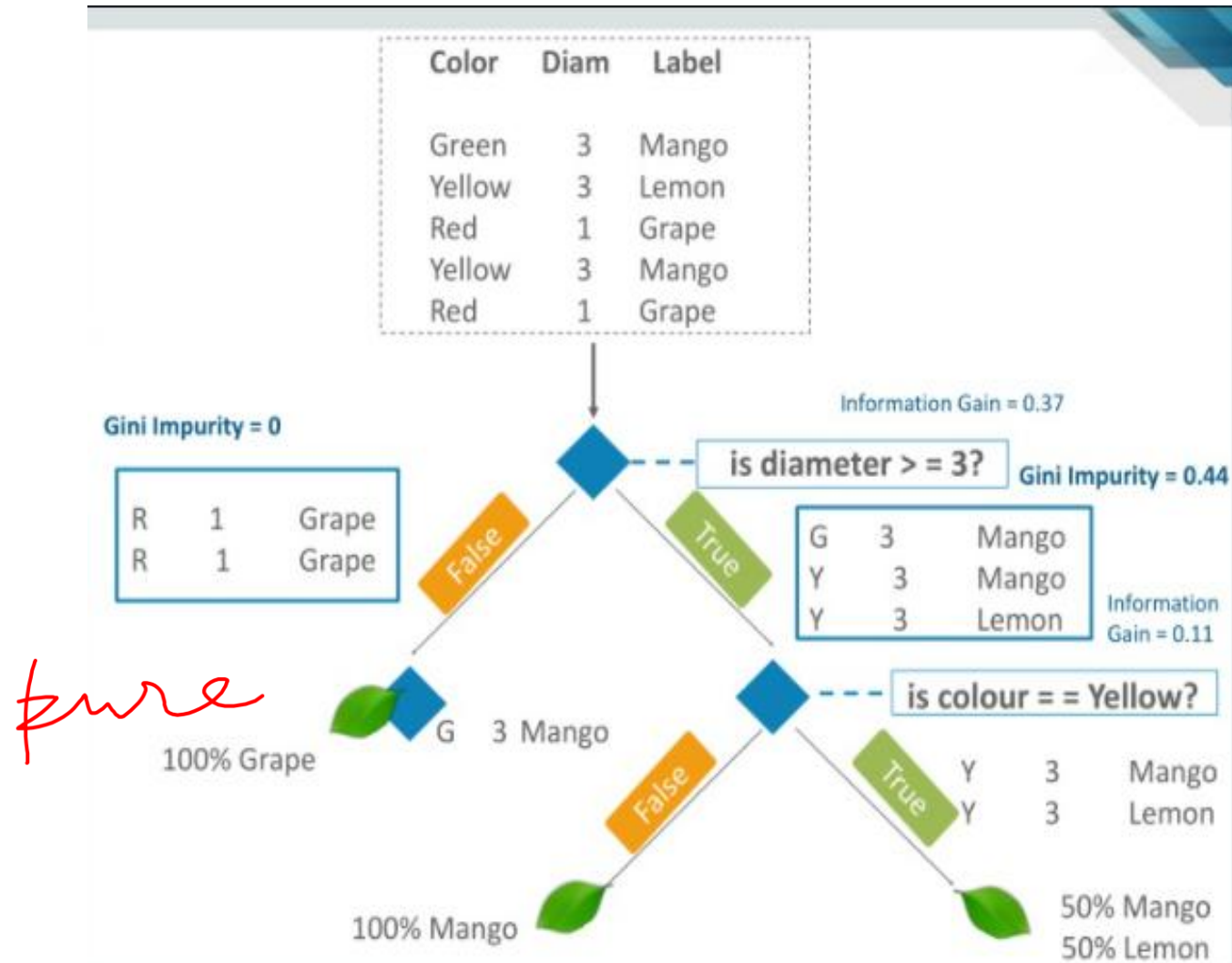
4. Stopping Condition: Splitting stops when:

- ✓ ■ A node becomes **pure** (i.e., all samples belong to the same class).
- ✓ ■ A predefined depth limit is reached.
- ✓ ■ The minimum number of samples per node is too small for further splitting.

5. Prediction (Leaf Nodes)

- ✓ ■ The final nodes (leaf nodes) hold the predicted class (for classification) or value (for regression).
- ✓ ■ For new input data, the decision tree follows the path based on the feature conditions until it reaches a leaf node.

Example of Creation of decision Tree



How to Create Decision Tree?

Which one among them
should you pick first?

Answer: Determine the
attribute that best
classifies the training data

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

How to Create Decision Tree?

But How do we choose
the best attribute?

Or

How does a tree decide
where to split?

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

How does a tree decide where to split?

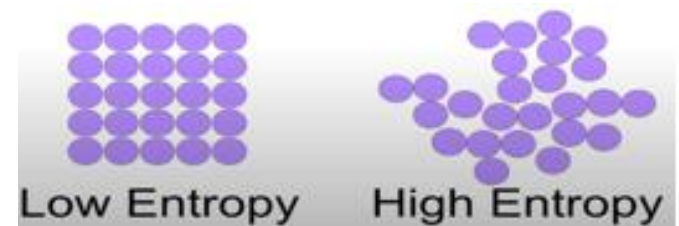
- Decision Trees are classification models that split data into nodes based on feature values.
- *To determine the best split, they rely on impurity metrics that evaluate how mixed a node's class distribution is.*
- **Gini Impurity and Entropy** are two measures used in decision trees to decide how to split data into branches.
- Both help determine how mixed or pure a dataset is, guiding the model toward splits that create cleaner groups.

Key Aspects of Splitting Decisions

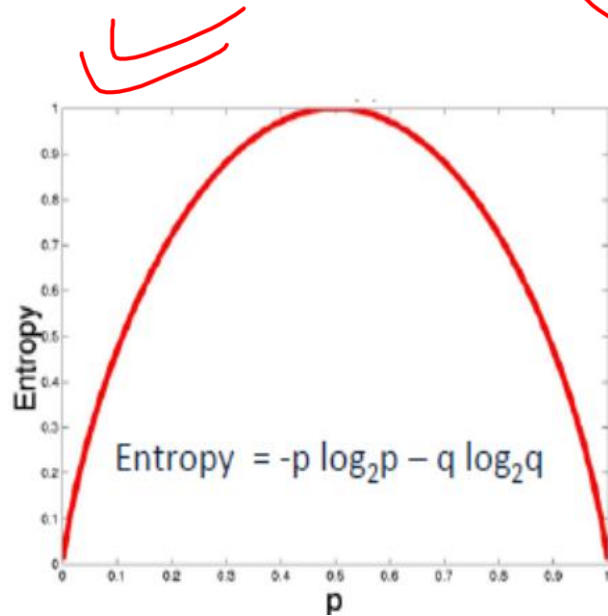
- 1) ✓ **Entropy & Information Gain:** Entropy Measures disorder/unpredictability.
 - The goal is to reduce entropy, with "Information Gain" determining the decrease in entropy after a split.
 - A higher information gain implies a better split.
 - Attribute with the highest information gain is considered the best.
- 2) **Gini Impurity:** Measures the probability of incorrect classification. The algorithm chooses the split that results in the lowest Gini impurity, aiming for a value of 0 (perfectly pure).

Entropy

- Entropy define randomness in the data.
- Entropy is the measure of impurity.
- **What is impurity:** impurity is the degree of randomness how random a data is so if the data is completely pure in that case the randomness equals 0.
 - If data is completely impure (meaning the classes are evenly mixed, such as a 50/50 split between two classes), the impurity value is at its **maximum**.
 - Impurity = 0 (Pure): All data in a node belongs to a single class (e.g., all 10 are "Yes").
 - Maximum Impurity (e.g., Gini 0.5 or Entropy 1): The data is evenly mixed (e.g., 5 "Yes" and 5 "No"), indicating maximum uncertainty.



Entropy



$\text{Log}(1)=0,$
 $\text{Log}(.5)=-1$

$$\text{Entropy}(s) = -P(\text{yes}) \log_2 P(\text{yes}) - P(\text{no}) \log_2 P(\text{no})$$

Where,

- S is the total sample space,
- $P(\text{yes})$ is probability of yes

If number of yes = number of no ie $P(S) = 0.5$

$$\Rightarrow \text{Entropy}(s) = 1$$

If it contains all yes or all no ie $P(S) = 1$ or 0

$$\Rightarrow \text{Entropy}(s) = 0$$

$\text{Log}(1)=0$

Information gain

- ✓ **Information gain** is the decrease in entropy.
- ✓ Information gain computes the difference between entropy before split and average entropy after split of the dataset based on given attribute values.

Information Gain

- Measures the reduction in entropy
- Decides which attribute should be selected as the decision node

If S is our total collection,

☆
Information Gain = $\text{Entropy}(S) - [(\text{Weighted Avg}) \times \text{Entropy}(\text{each feature})]$

Build a decision tree for our data set: Compute the Entropy of Entire Data Set

Out of 14 instances we have 9 YES and 5 NO

So we have the formula,

$$E(S) = -P(\text{Yes}) \log_2 P(\text{Yes}) - P(\text{No}) \log_2 P(\text{No})$$

$$E(S) = - (9/14) * \log_2 9/14 - (5/14) * \log_2 5/14$$

$$E(S) = 0.41 + 0.53 = 0.94$$

	outlook	temp.	humidity	windy	play
D1	sunny	hot	high	false	no
D2	sunny	hot	high	true	no
D3	overcast	hot	high	false	yes
D4	rainy	mild	high	false	yes
D5	rainy	cool	normal	false	yes
D6	rainy	cool	normal	true	no
D7	overcast	cool	normal	true	yes
D8	sunny	mild	high	false	no
D9	sunny	cool	normal	false	yes
D10	rainy	mild	normal	false	yes
D11	sunny	mild	normal	true	yes
D12	overcast	mild	high	true	yes
D13	overcast	hot	normal	false	yes
D14	rainy	mild	high	true	no

Our data set which consists of 14 different instances out of which we have 9 “Yes” and 5 “No”

Which Node to be selected as Root Node?

Outlook?

Temperature?

Humidity?

Windy?

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

Information Gain = Entropy(S) – [(Weighted Avg) x Entropy(each feature)]

Entropy(s) = - P(yes) log₂ P(yes) – P(no) log₂ P(no)

Which Node to be selected as Root Node: Outlook?

- Now you have to select that out of outlook temperature humidity and Windy, which the node should you select as the root node big question, right?
- How will you decide that?
 - One by one you have to calculate the entropy and information gain for all of the different nodes.
 - Outlook:
 - Outlook** has three different parameters sunny overcast and rainy
 - When it is sunny, how many number of Yes and No are there. : 2 yes and 3 no
 - In case of overcast: all Yes (if overcast then will surely go to play)
 - Raining: 3Yes and 2 No

- Calculate the entropy for each feature

$$E(\text{Outlook} = \text{Sunny}) = -2/5 \log_2 2/5 - 3/5 \log_2 3/5 = 0.971$$

$$E(\text{Outlook} = \text{Overcast}) = -1 \log_2 1 - 0 \log_2 0 = 0$$

$$E(\text{Outlook} = \text{Rainy}) = -3/5 \log_2 3/5 - 2/5 \log_2 2/5 = 0.971$$

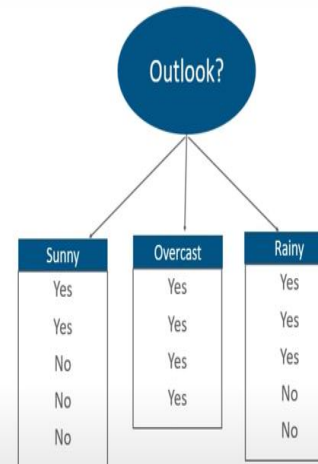
Information from outlook,

$$I(\text{Outlook}) = 5/14 \times 0.971 + 4/14 \times 0 + 5/14 \times 0.971 = 0.693$$

Information gained from outlook,

$$\text{Gain}(\text{Outlook}) = E(S) - I(\text{Outlook})$$

$$0.94 - 0.693 = 0.247$$



outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

Which Node to be selected as Root Node: Windy?

$$E(\text{Windy} = \text{True}) = 1$$

$$E(\text{Windy} = \text{False}) = 0.811$$

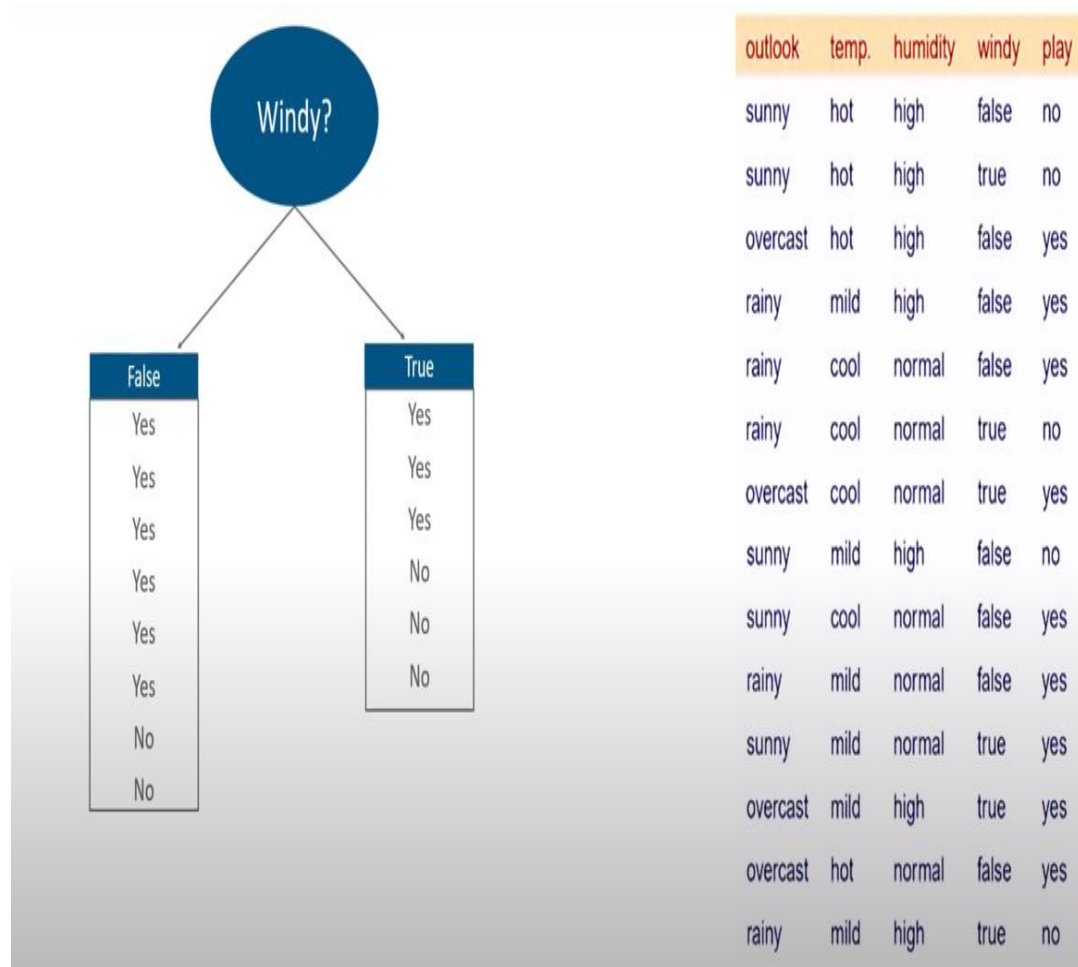
Information from windy,

$$I(\text{Windy}) = 8/14 \times 0.811 + 6/14 \times 1 = 0.892$$

Information gained from outlook,

$$\text{Gain}(\text{Windy}) = E(S) - I(\text{Windy})$$

$$0.94 - 0.892 = 0.048$$



Similarly we calculate for next two attributes.

Which node to select as root node

Outlook:

Info

0.693

Gain: $0.940 - 0.693$

0.247

Temperature:

Info

0.911

Gain: $0.940 - 0.911$

0.029

Humidity:

Info

0.788

Gain: $0.940 - 0.788$

0.152

Windy:

Info

0.892

Gain: $0.940 - 0.982$

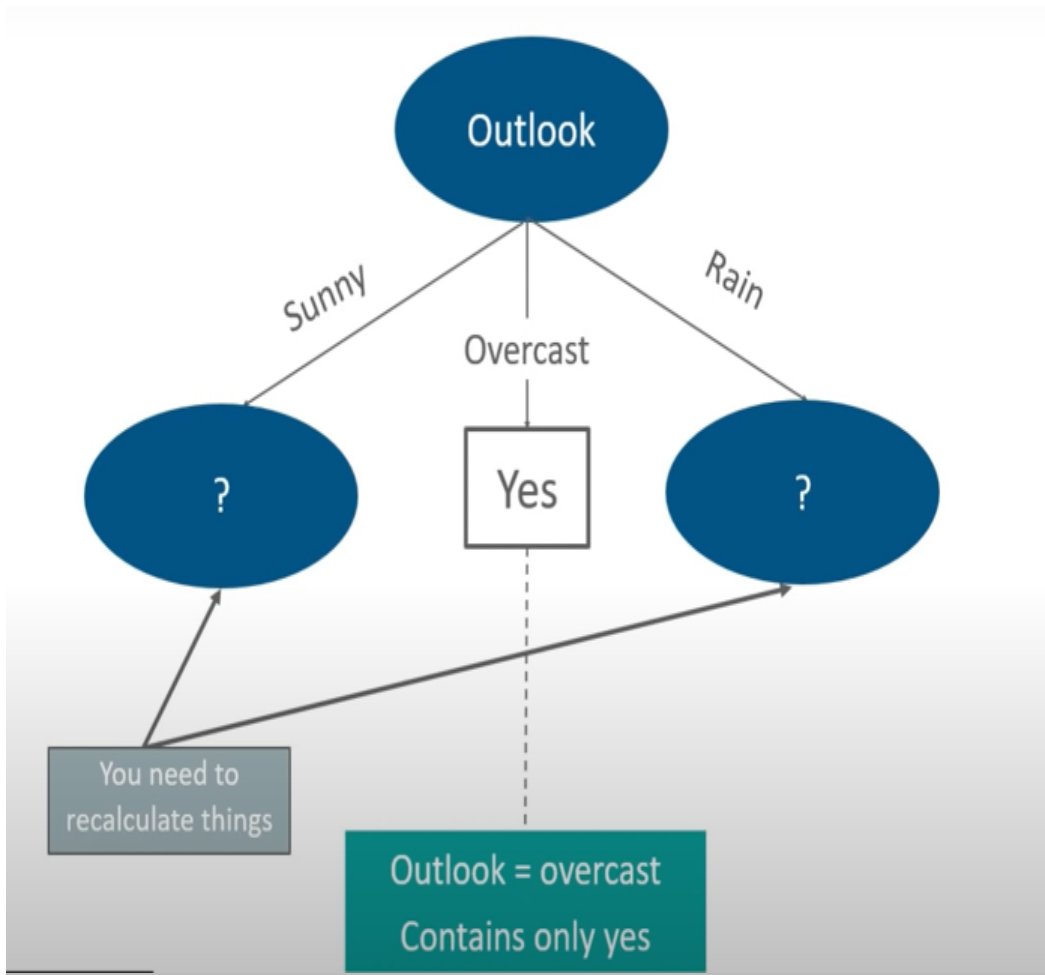
0.048

outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

Since Max gain = 0.247,

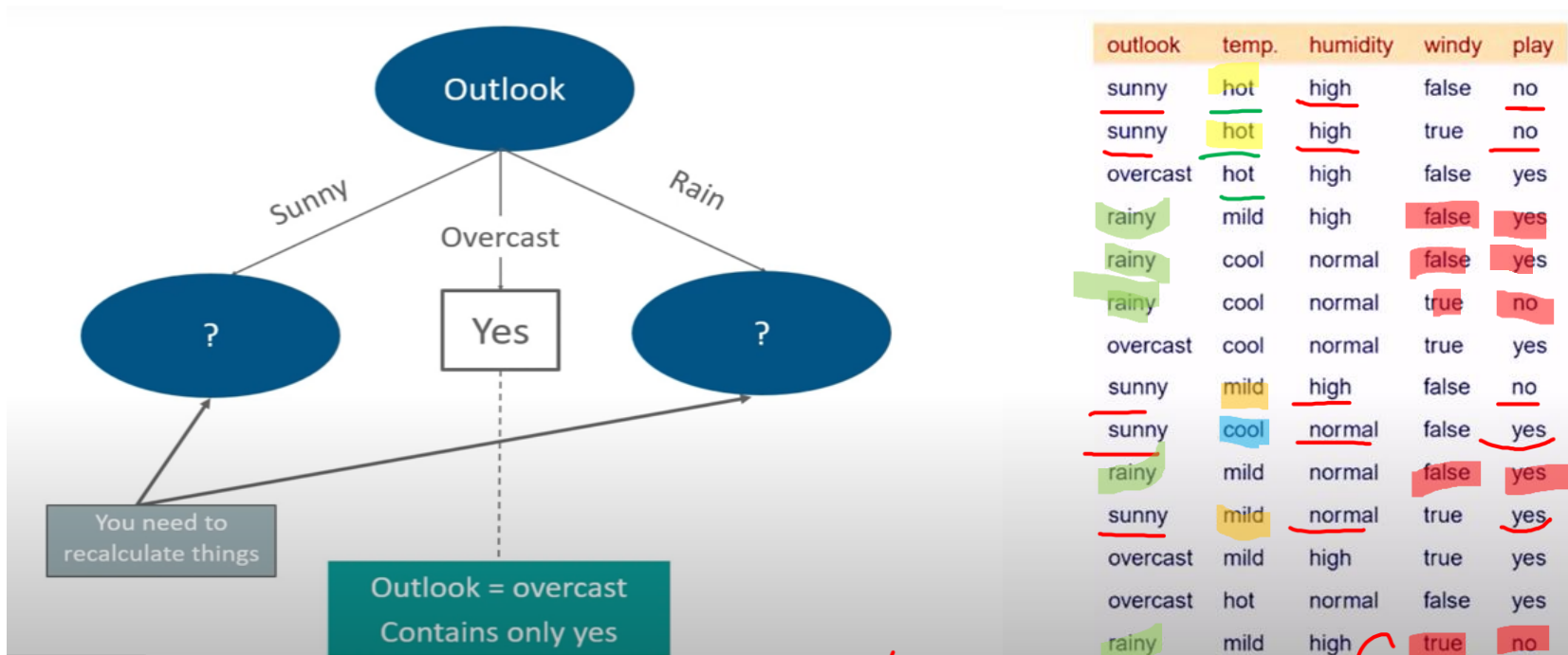
Outlook is our ROOT Node

Which Node to select further?



outlook	temp.	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

Which Node to select further?



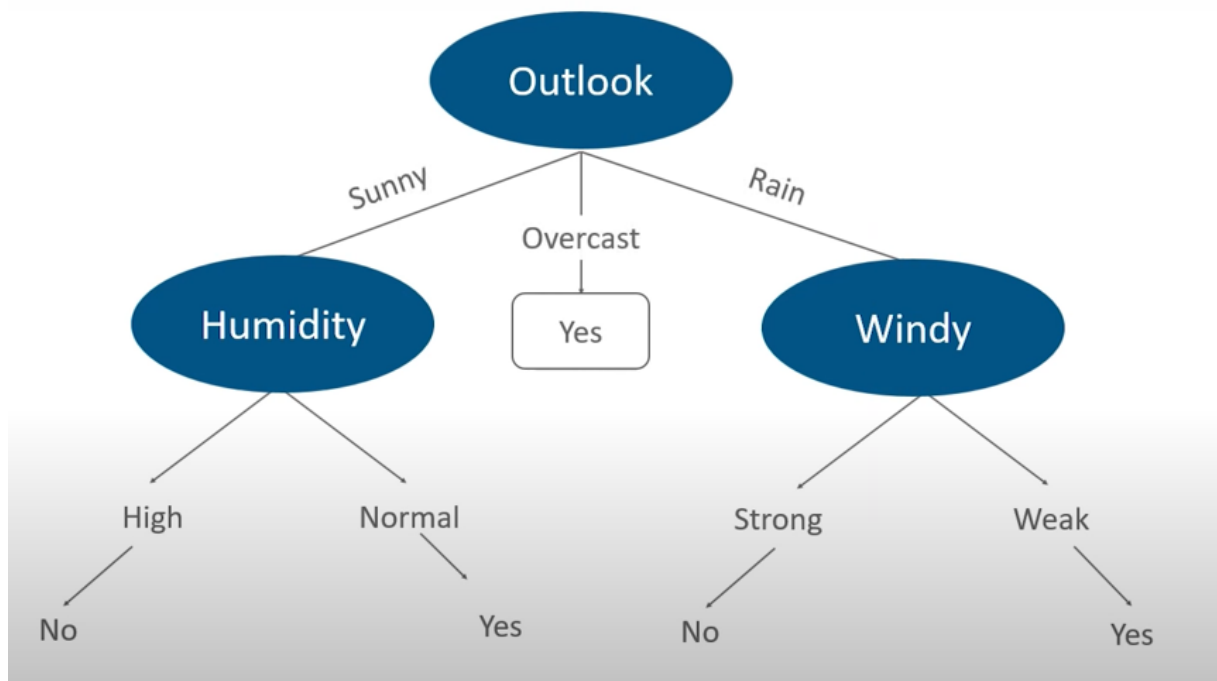
$$\begin{aligned} \text{Gain}(S_{\text{sunny}}, \text{Humidity}) &= 0.970 - (3/5)0.0 - 2/5(0.0) = 0.970 \\ \text{Gain}(S_{\text{sunny}}, \text{Temp.}) &= 0.970 - (2/5)0.0 - 2/5(1.0) - (1/5)0.0 = 0.570 \\ \text{Gain}(S_{\text{sunny}}, \text{Wind}) &= 0.970 - (2/5)1.0 - 3/5(0.918) = 0.019 \end{aligned}$$

Humidity (5 rows)

High 3
 no
no
no

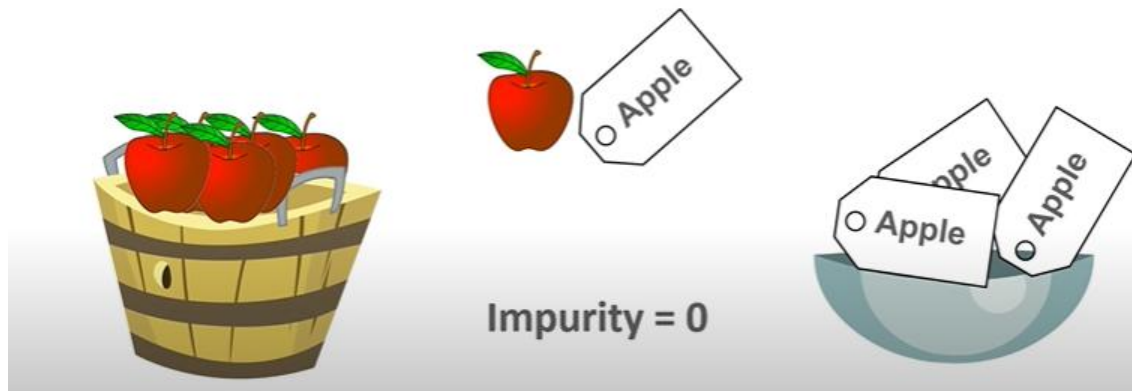
normal 1
 yes
yes

Complete Decision Tree for the example



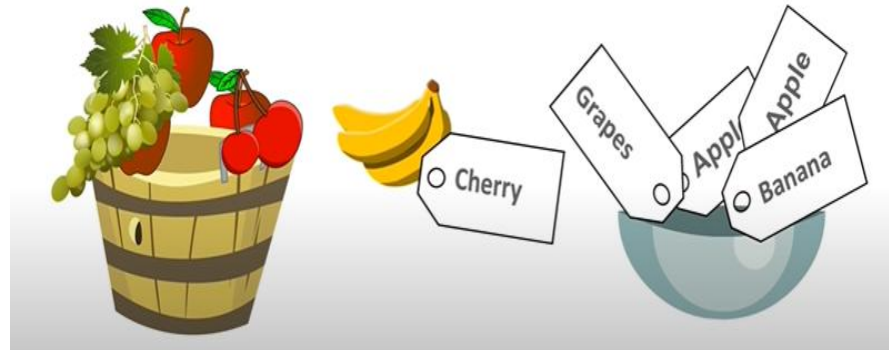
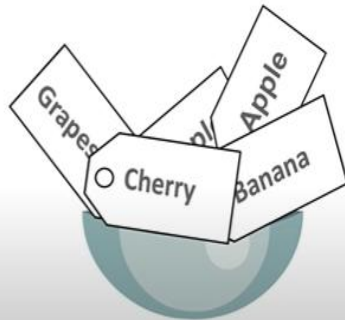
Impurity

- You have a basket full of apples and another Bowl which is full of same label, which says Apple.
- If you are asked to pick one item from each basket and bowl then the probability of getting the Apple and it's correct label is 1
- So in this case, you can say that impurity is 0 all right.



Impurity

- If there are 4 different fruits in the basket and 4 different labels in the ball, then the probability of matching the fruit to a label is obviously not 1 it's something less than that.
- So in this case, I would say that impurities is nonzero.



Gini Impurity

- The **Gini index**, or **Gini impurity**, is a metric used in the construction of decision trees to measure the **impurity** or disorder of a node.
- It quantifies the probability of incorrectly classifying a randomly chosen element in the dataset if it were labeled according to the class distribution within that node.
- Gini Impurity checks how often a randomly selected sample would be mislabeled if assigned by class probability.
- It is computationally simple and used in tree-based classifiers.
- **Purpose:** The primary goal is to find the best feature and split point that minimizes the Gini index (maximizes purity) at each step of building the tree.

Gini Impurity

- If there are n classes, Gini impurity is :

$$Gini(p) = 1 - \sum_{i=1}^n p_i^2$$

Where p_i is the probability of class i in a node.

- **Properties:**

- Lower values indicate cleaner and more homogeneous nodes.
- Nodes become pure when all samples belong to one class.
- Slightly biased toward dominant classes during split selection.

Example: how to calculate Gini Impurity for Binary classification

Let's take an illustration and assume we have a node with:

- 3 instances of Class A
- 2 instances of Class B

Then:

$$p_A = \frac{3}{5}, \quad p_B = \frac{2}{5}$$

$$\begin{aligned} Gini &= 1 - (p_A^2 + p_B^2) \\ &= 1 - \left(\frac{9}{25} + \frac{4}{25} \right) = 1 - \frac{13}{25} \\ &= \frac{12}{25} = 0.48 \end{aligned}$$

Calculating Gini Gain

- ✓ 1. Gini Gain quantifies the reduction in impurity achieved by splitting a node on a particular feature. The process is as follows:
- ✓ 2. Calculate the Gini Impurity of the parent node before any split.
- ✓ 3. Calculate the Gini Impurity of each potential child node resulting from the split.
- ✓ 4. Calculate the weighted average impurity of the child nodes. The weight for each child node is the proportion of data points it contains compared to the parent node.
- ✓ 5. Subtract the weighted child impurity from the parent's original impurity to get the Gini Gain for that specific feature.
- ✓ 6. The formula for Gini Gain for a split on feature is:

$$\text{Gini Gain}(A) = \text{Gini Impurity}(\text{Parent}) - \text{Weighted Gini Impurity}(\text{Children})$$

- ✓ 7. The decision tree algorithm evaluates the Gini Gain for all available features and selects the feature that yields the highest Gini Gain for the split at that node.

Gini Impurity

- ✓ **Range: 0 to 0.5 (for binary classification):** Gini ranges from 0 for Pure node (only one class, e.g., all are "Yes") to 0.5 for Maximum impurity (equal mix of two classes, e.g., 50% "Yes" and 50% "No"). In multiclass classification, the maximum Gini can be higher but still ≤ 1 .
- ✓ **Faster to compute than entropy:** Gini uses only squares of probabilities (no logarithms), which is computationally lighter and faster.
- ✓ **Used in CART Trees (Classification and Regression Trees):** CART is the algorithm behind Decision Tree Classifier in scikit-learn. It uses Gini by default to evaluate splits.

Gini Impurity

- **Sensitive to class imbalance:** If one class dominates (e.g., 95% "A" and 5% "B"), Gini might not strongly penalize impurity, leading to biased trees. May prefer splits favoring the majority class.
- **No logarithmic calculation:** As mentioned, Gini avoids the computational burden of logs. This makes it faster, especially when training on large datasets.
- **Gini = 0 means Pure Node:** A leaf node with Gini = 0 means it contains only one class. The goal is that we want our leaf nodes to be as pure as possible for high model accuracy.

Splitting Criteria In Decision Tree

In decision trees, splitting criteria help decide which feature to split on at each node. The two most common criteria are:

Gini Index

$$I_G = 1 - \sum_{j=1}^c p_j^2$$

p_j : proportion of the samples that belongs to class c for a particular node

Entropy

$$I_H = - \sum_{j=1}^c p_j \log_2(p_j)$$

p_j : proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes ($p \neq 0$) The entropy is 0 if all samples at a node belong to the same class.

When To Prefer Which Metric?

Scenario	Gini Impurity	Entropy
Training Speed	Faster computation since it avoids <u>log operations</u>	Slightly slower due to <u>logarithmic calculations</u>
Dataset Size	Efficient for <u>large, high-dimensional datasets</u>	Useful for structured <u>datasets with balanced classes</u>
Common Usage	Gini is used in the <u>CART</u> (Classification and Regression Trees) algorithm, while The Python library scikit-learn uses Gini by default.	Information Gain (which relies on entropy) is used in algorithms like ID3 and <u>C4.5.</u>

CART algorithm

- The **CART algorithm** is a specific type of decision tree algorithm.
- To break a dataset into smaller, meaningful groups, CART (Classification and Regression Tree) is used which builds a decision tree that predicts outcomes for **both classification and regression tasks**.
- It works by splitting data based on rules that reduce error at each step.
 - Uses **binary** splits at every node.
 - Simple to understand and interpret.
 - Works for both **numeric and categorical data**.
 - Forms the base of advanced models like **Random Forest, Gradient Boosting and XGBoost**.

How CART Builds a Decision Tree

- CART (Classification and Regression Trees) constructs a decision tree by recursively splitting the dataset based on the feature and threshold that produce the highest reduction in impurity (for classification) or error (for regression)
- **Step 1: Evaluate Best Split for Each Feature**
 - Check all features and their possible split values.
 - Compute impurity or error.
 - Identify the split that gives the maximum reduction in impurity or error.
- **Step 2: Select the Optimal Split**
 - Compare impurity decrease for all features.
 - Choose the feature and threshold pair with the best score.
 - This becomes the splitting rule for the current node.

How CART Builds a Decision Tree

- **Step 3: Create Binary Child Nodes**
 - Split the dataset into Left Node (values less than or equal to threshold) and Right Node (values greater than or equal to threshold).
 - Assign corresponding samples to each child node.
- **Step 4: Apply Recursive Splitting**
 - Repeat the same process for each child node.
 - Continue until a stopping condition is met.

CART for Classification

- CART is used for classification tasks when the output variable is categorical.
- **How it Works**
 - Recursively splits the dataset to increase class purity at each step.
 - Uses Gini Impurity to select the best feature and threshold for splitting.
 - Continues splitting until a stopping rule is met such as maximum tree depth or minimum required samples.
 - CART chooses splits that produce the purest possible child nodes.

CART for Regression

- CART is used for regression tasks where the output variable is numerical.
- **How it Works**
 - Splits the data to minimize residual error between actual and predicted values.
 - Uses Mean Squared Error (MSE) or Residual Sum of Squares (RSS) as the splitting criterion.
 - Each leaf node stores the mean of the target values within that node, which becomes the prediction for new samples.
 - This helps CART create a tree that provides the lowest possible prediction error.

Common Stopping or Pruning Criteria Used in CART

- **Maximum tree depth reached:** The tree stops growing after a predefined maximum depth.
- **Minimum samples required to split a node:** Splitting stops when a node has fewer samples than the minimum required.
- **No further improvement in impurity:** If a split does not reduce impurity or error, CART stops splitting.
- **Node becomes pure:** When all samples belong to one class, no further split is needed.

Hyperparameters in CART

- CART provides several hyperparameters to control the tree structure, prevent overfitting and improve model performance. Important hyperparameters include:
 - **max_depth**: Controls the maximum depth of the tree, preventing it from growing too deep and overfitting the data.
 - **min_samples_split**: Specifies the minimum number of samples required to split an internal node; higher values make the tree more conservative.
 - **min_samples_leaf**: Determines the minimum number of samples allowed in a leaf node, helping avoid very small or unstable leaves.
 - **max_features**: Defines the number of features to consider while looking for the best split; useful for reducing computation and variance.
 - **criterion**: Specifies the function used to evaluate the split gini or entropy for classification and MSE for regression.

Difference between General Decision Tree and CART algorithm

Feature 	General Decision Tree (e.g., ID3, C4.5)	CART (Classification And Regression Trees)
Problem Type	Often specialized for classification (ID3) or can handle both (C4.5).	Handles both Classification and Regression tasks.
Splitting Criteria	Uses various metrics like Information Gain (ID3, C4.5) or Gain Ratio (C4.5).	Uses Gini Impurity for classification and variance reduction (Mean Squared Error) for regression.
Tree Structure	Can produce multi-way splits, where a single node branches into more than two child nodes.	Always produces binary splits , meaning each internal node splits into exactly two child nodes.
Data Handling	ID3 primarily works with categorical data; C4.5 and C5.0 handle continuous features better.	Naturally handles both categorical and continuous data types effectively.
Pruning	Pruning methods vary by algorithm (e.g., C4.5 has advanced pruning, ID3 lacks it).	Uses a specific post-pruning approach to prevent overfitting.

Advantages of Decision Trees

- **Easy to Understand**: Decision Trees are visual which makes it easy to follow the decision-making process.
- **Versatility**: Can be used for both classification and regression problems.
- **No Need for Feature Scaling**: Unlike many machine learning models, it don't require us to scale or normalize our data.
- **Handles Non-linear Relationships**: It capture complex, non-linear relationships between features and outcomes effectively.
- **Interpretability**: The tree structure is easy to interpret helps in allowing users to understand the reasoning behind each decision.
- **Handles Missing Data**: It can handle missing values by using strategies like assigning the most common value or ignoring missing data during splits.

Disadvantages of Decision Trees

- **Overfitting**: They can overfit the training data if they are too deep which means they memorize the data instead of learning general patterns. This leads to poor performance on unseen data.
- **Instability**: It can be unstable which means that small changes in the data may lead to significant differences in the tree structure and predictions.
- **Bias towards Features with Many Categories**: It can become biased toward features with many distinct values which focuses too much on them and potentially missing other important features which can reduce prediction accuracy.
- **Difficulty in Capturing Complex Interactions**: Decision Trees may struggle to capture complex interactions between features which helps in making them less effective for certain types of data.
- **Computationally Expensive for Large Datasets**: For large datasets, building and pruning a Decision Tree can be computationally intensive, especially as the tree depth increases.

Applications of Decision Trees

- Decision Trees are used across various fields due to their simplicity, interpretability and versatility lets see some key applications:
- ✓ • **Loan Approval in Banking:** Banks use Decision Trees to assess whether a loan application should be approved. The decision is based on factors like credit score, income, employment status and loan history. This helps predict approval or rejection helps in enabling quick and reliable decisions.
- ✓ • **Medical Diagnosis:** In healthcare they assist in diagnosing diseases. For example, they can predict whether a patient has diabetes based on clinical data like glucose levels, BMI and blood pressure. This helps classify patients into diabetic or non-diabetic categories, supporting early diagnosis and treatment.
- ✓ • **Predicting Exam Results in Education:** Educational institutions use to predict whether a student will pass or fail based on factors like attendance, study time and past grades. This helps teachers identify at-risk students and offer targeted support.

Applications of Decision Trees

- **Customer Churn Prediction:** Companies use Decision Trees to predict whether a customer will leave or stay based on behavior patterns, purchase history, and interactions. This allows businesses to take proactive steps to retain customers.
- **Fraud Detection:** In finance, Decision Trees are used to detect fraudulent activities, such as credit card fraud. By analyzing past transaction data and patterns, Decision Trees can identify suspicious activities and flag them for further investigation.

Pruning in Decision Tree

- ✓ • Pruning is a model optimization technique used to prevent overfitting in decision trees by removing branches that provide little to no predictive power.
- ✓ • A fully grown tree often captures "noise" and accidental patterns in the training data, leading to a complex model that performs poorly on new, unseen data.
- ✓ • Pruning simplifies the tree, making it smaller, more interpretable, and better at generalising.
- Two strategies for pruning:
 - **Pre-Pruning (Early Stopping)**
 - **Post-Pruning (Backward Pruning)**

Core Pruning Strategies

- Pruning techniques are broadly classified into two main categories based on when the pruning occurs:
 - **Pre-Pruning (Early Stopping):** This stops the tree-building process before it becomes too complex. Growth is halted during construction if certain criteria are met, such as:
 - **Maximum Depth:** Limiting how many levels the tree can have.
 - **Minimum Samples per Split:** Requiring a minimum number of data points to allow a further split.
 - **Minimum Samples per Leaf:** Ensuring every final node contains at least a specific number of instances.
 - **Information Gain Threshold:** Stopping if a split doesn't improve the model's accuracy beyond a certain limit.
 - **Risk:** It can lead to underfitting by stopping too early, potentially missing important relationships that only become clear with further splits.

Advantages and Disadvantages of Pre- Pruning

- **Advantages of Pre-Pruning:**

- **Reduces Overfitting:** By stopping the tree's growth early, it avoids capturing noise in the data.
- **Faster Training:** Pre-pruning results in smaller trees, which are faster to train and easier to interpret.

- **Disadvantages of Pre-Pruning:**

- **Risk of Underfitting:** If the stopping criteria are too strict, the tree might be too shallow to capture important patterns in the data, leading to underfitting.

Post-Pruning (Backward Pruning)

- **Post-Pruning (Backward Pruning):** This allows the tree to grow to its full depth and then removes non-significant branches. It is generally more robust but computationally expensive.
- Common methods include:
 - **1. Cost-Complexity Pruning (CCP):** Minimal cost-complexity pruning (CCP) works by recursively finding the "weakest link" in the tree, which is a node that, when pruned, leads to the smallest increase in total impurity relative to the number of leaves removed. This process generates a sequence of pruned trees, each associated with a specific `ccp_alpha` value.
 - **ccp_alpha.** As `ccp_alpha` increases, more nodes are pruned.
 - **Purpose of `ccp_alpha`:** The primary goal of `ccp_alpha` is to simplify the tree structure so that it generalizes better to unseen data, thereby improving testing accuracy and reducing variance.
 - This method, used in the CART algorithm, balances tree complexity and accuracy by penalizing larger trees with a parameter

Post-Pruning (Backward Pruning)

2. Reduced Error Pruning (REP): method for reducing the complexity of a fully grown decision tree to prevent overfitting, using a separate validation dataset to guide the pruning process. The key steps are as follows:

- **Data Split:** The initial dataset is typically divided into three separate sets: a training set to build the initial full tree, a validation (or pruning) set to make pruning decisions, and a test set to evaluate the final model's performance on unseen data.
- **Bottom-Up Evaluation:** Starting from the leaf nodes, every non-leaf node is considered as a potential candidate for pruning.
- **Replacement and Evaluation:** For a given node, the entire subtree rooted at that node is removed and replaced by a single leaf node. This new leaf node is assigned the most frequent class label among the examples in the validation set that reach that node.
- **Accuracy Check:** The accuracy of the modified tree on the validation set is then compared to the accuracy of the original tree. If the new, pruned tree performs no worse than the original on the validation set, the change is considered beneficial and is kept. If pruning results in a decrease in validation accuracy, the original subtree is restored.
- **Iteration:** This process is repeated iteratively in a bottom-up fashion for all nodes until no further pruning improves or maintains the validation accuracy.

Advantages and Disadvantages of Post- Pruning

- **Advantages of Post-Pruning:**

- **Better Generalization:** Since the tree is first grown fully and then pruned, post-pruning allows the model to consider all possible patterns before discarding irrelevant or weak patterns.
- **Avoids Overfitting:** Post-pruning typically results in a simpler model that generalizes better to unseen data.

- **Disadvantages of Post-Pruning:**

- **Slower:** Post-pruning can be computationally expensive because it requires growing the tree fully first and then iterating through the pruning process.
- **Validation Set Required:** Requires a validation set to determine which branches to prune, reducing the amount of data available for training the model.

Pre-pruning verses Post-pruning

Aspect	Pre-Pruning	Post-Pruning
✓ Tree Growth	Tree stops growing early based on conditions	Tree grows fully, and then unnecessary branches are pruned
✓ Risk	Risk of underfitting if stopping criteria are too strict	Risk of overfitting initially, but pruning reduces complexity
✓ Computation	Faster, as the tree is not grown fully	Slower, requires growing and pruning the tree
✓ Control	Requires careful tuning of stopping criteria	More control over pruning decisions based on validation results
✓ Interpretability	Smaller and simpler tree from the start	Tree is simplified after pruning, balancing complexity and accuracy

Ensemble Learning

- Ensemble learning combines multiple machine learning models—often weak learners—to create a single, stronger, and more robust predictive model, improving accuracy and reducing overfitting.
- **Multiple Models:** Uses many small models together
- **Better Accuracy:** Combined results improve performance
- **Reduced Errors:** Mistakes of one model are balanced by those of others
- **Simple Idea:** Like taking advice from a group instead of one person

Ensemble Learning

- In machine learning, **ensemble techniques** are used to improve accuracy, reduce variance, and enhance generalization to unseen data.
- Instead of relying on a single model, ensemble methods leverage the collective power of several models to make more robust predictions.
- Two of the most popular ensemble techniques are **bagging** and **boosting**, both of which are widely used to enhance the performance of models, especially decision trees.

Key Ensemble Learning Techniques

✓ • **Bagging (Bootstrap Aggregating):** This technique reduces variance by training multiple models (usually decision trees) in parallel on different subsets of the training data, then averaging their predictions (regression) or taking a majority vote (classification).

• Example: Random Forest.

✓ • **Boosting:** A sequential technique that reduces bias and variance by training models iteratively. Each new model attempts to correct the errors made by the previous ones, focusing on harder-to-predict data points.

• Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM.

• St

Why Use Ensemble Methods?

- **Improved Performance:** Generally offers higher accuracy than single models, commonly winning Kaggle competitions.
- **Reduced Overfitting:** Bagging helps prevent overfitting by averaging multiple models.
- **Reduced Bias:** Boosting minimizes errors by focusing on misclassified instances.
- **Robustness:** Combines diverse models to improve generalization.

Advantages & Disadvantages of Ensemble Learning

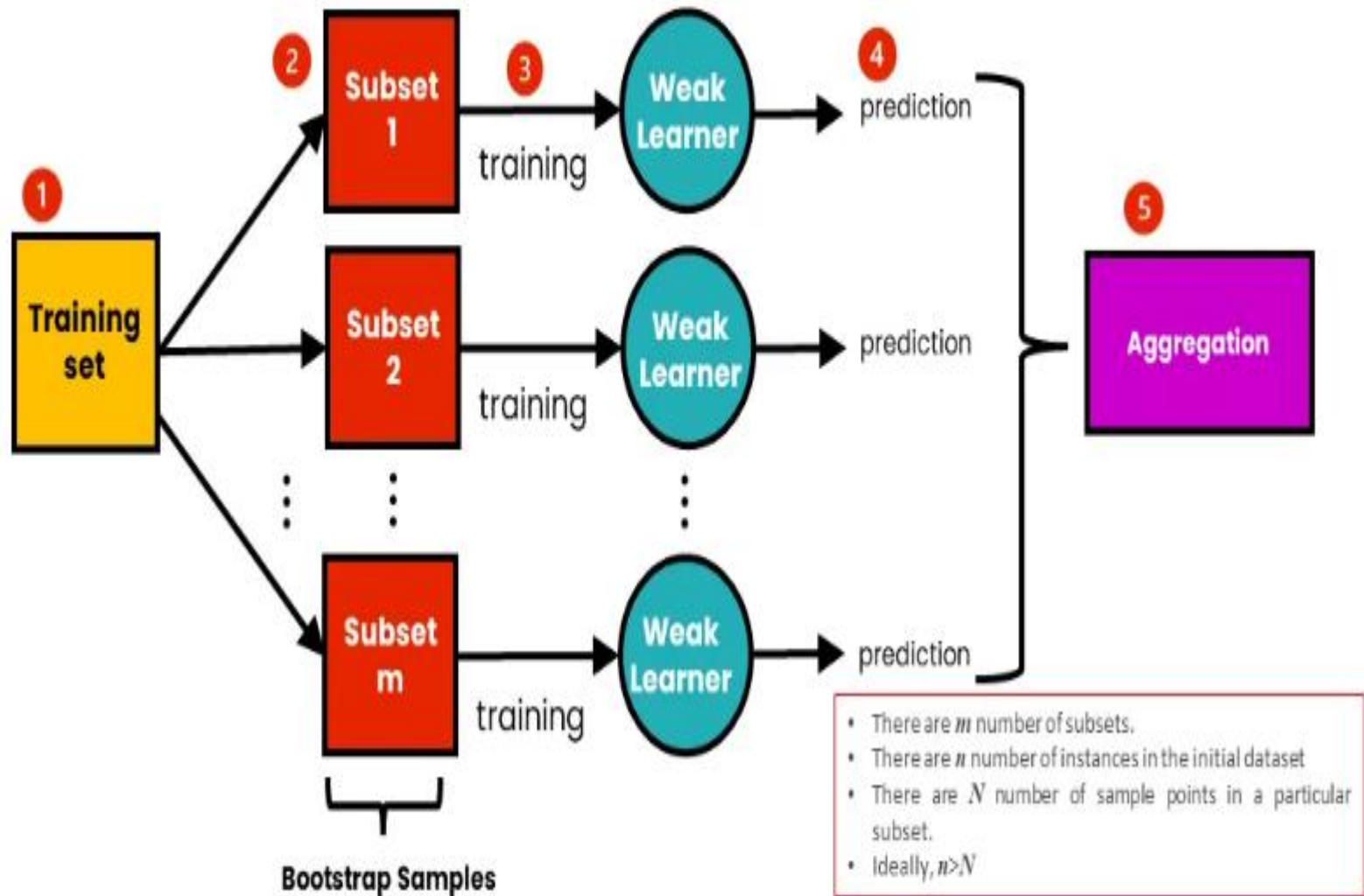
Advantages & Disadvantages

- **Pros:** Highly accurate and robust.
- **Cons:** Increased computational cost, complexity in training and maintenance, and decreased interpretability.

Bagging (Bootstrap Aggregating)

- **How it works:** Multiple versions of the same model type are trained independently and in parallel on different random subsets of the data (created via **bootstrapping**, or sampling with replacement).
- **Goal:** To reduce **variance** and prevent overfitting by averaging the results.
- **Example: Random Forest**, which uses many decision trees to make a collective prediction

The Process of Bagging (Bootstrap Aggregation)



Steps in Bagging

- The following steps are involved in implementing bagging in machine learning:

1. **Select a Base Model:** Select a base model that is known to perform well on a particular task. The base model should have a high variance, which indicates that it overfits the data.

2. **Bootstrapped Sampling:** Using random sampling with replacement, create numerous subsets of the training data.

- The ensemble's desired number of base models is equal to the number of subgroups formed.
- Each subset should be the same size as the original training data
- Each data point should have an equal chance of inclusion in each subset. This is known as bootstrapped sampling, and it aids in the creation of variety among base models.

3. **Base Model Training:** Independently train each base model on its bootstrapped subset of training data. Fitting the base model to the subset of data and deriving the model parameters or weights is what this entails.

Steps in Bagging

✓ 4. ***Combination of Prediction Models:*** Once all of the base models have been trained, they may be utilized to make predictions on the test data.

- To create the final prediction, the predictions of the base models can be integrated using a voting or averaging approach.
- The most popular strategy for classification issues is to employ majority voting, in which the class with the most votes from the base models is chosen as the final prediction.
- The predictions for regression issues can be averaged to give the final forecast.

✓ 5. ***Evaluation and Performance Analysis:*** Depending on the kind of problem, evaluate the performance of the bagged ensemble model using appropriate evaluation metrics such as accuracy, precision, recall, F1-score, or RMSE. To evaluate the improvement in model performance, compare the performance of the bagged ensemble model to the performance of the base model.

Steps in Bagging

6. To further optimize model performance, fine-tune the hyperparameters of the basic model and the bagging ensemble. Experiment with different hyperparameter values to identify the optimum combination for the given problem, such as the number of base models in the ensemble, the sample ratio for bootstrapped sampling, and the type of prediction combination scheme.

Random Forest

The random forest is a powerful and popular ensemble learning algorithm in machine learning that builds multiple decision trees during training and merges their predictions to produce a more accurate and stable final result.

It is widely used for both classification (predicting categories via majority vote) and regression (predicting numerical values via averaging) tasks.

Random Forest

How the Algorithm Works

The strength of the random forest lies in the diversity of its individual trees, achieved through two key randomization methods:

✓ **1. Bootstrap Aggregating (Bagging):** Each tree in the forest is trained on a random subset of the original data, selected *with replacement*. This means some data points may be used multiple times in one tree's training, while others (known as "out-of-bag" samples) are left out.

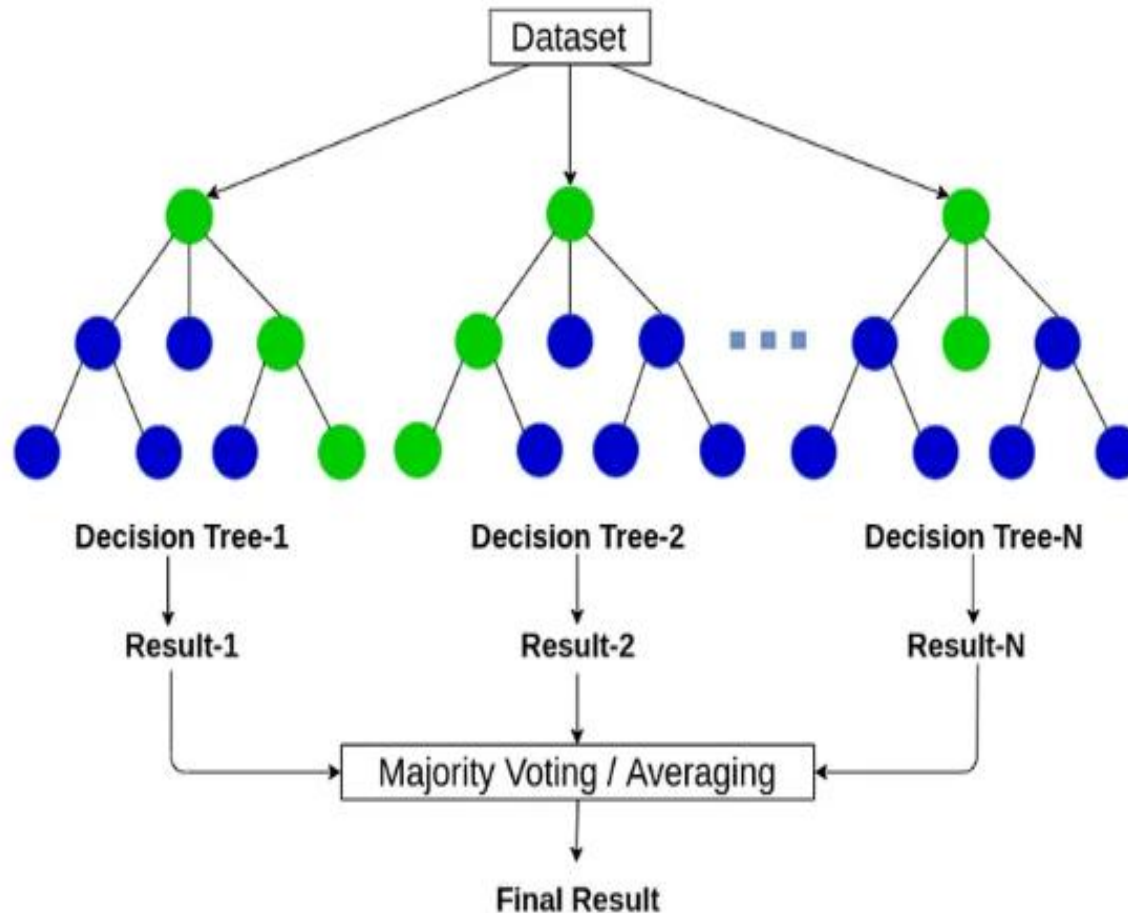
✓ **2. Feature Randomness:** At each node of a tree, only a random subset of the available features is considered when determining the best split. This prevents a single, strong feature from dominating all trees, ensuring they remain uncorrelated and diverse.

Random Forest

Once all trees are built:

- ✓ • For a **classification** problem, each tree "votes" for a class, and the class with the most votes becomes the forest's final prediction.
- ✓ • For a **regression** problem, the predictions from all trees are averaged to produce the final continuous output.

Random Forest



Sampling Techniques in Random Forest

The random forest algorithm uses two primary sampling techniques to ensure the individual decision trees in the ensemble are diverse and decorrelated: **bootstrap aggregating (bagging)** and **random feature selection**. This approach helps to reduce the model's variance and prevent overfitting.

✓ 1. Bootstrap Aggregating (Bagging)

Bagging involves creating multiple random samples of the original training data, with replacement, to train each individual decision tree.

- ✓ • Sampling with Replacement: This means that after a data point is selected for a sample, it is "returned" to the original dataset and can be selected again for the same or other samples.
- ✓ • Sample Size: Each bootstrap sample is typically the same size as the original training set.
- ✓ • Out-of-Bag (OOB) Samples: Due to sampling with replacement, each tree is trained on only about two-thirds of the original data. The remaining one-third of the data, the out-of-bag samples, can be used as an internal validation set to estimate the model's performance without the need for a separate test set.

Sampling Techniques in Random Forest

2. Random Feature Selection

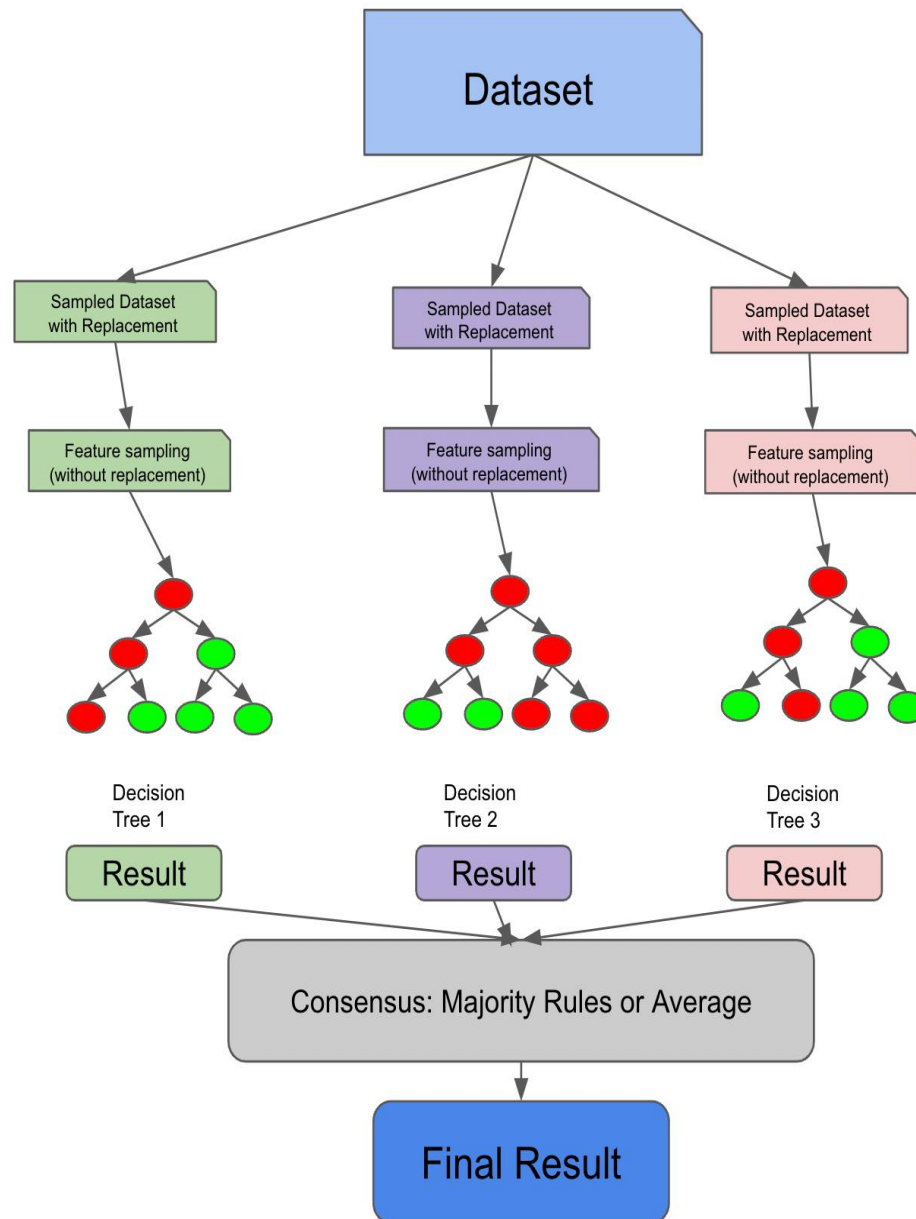
In addition to sampling the data (rows), the random forest algorithm also injects randomness by selecting a random subset of features (columns) at each split within a decision tree.

- **Subset of Features:** Instead of considering all available features to find the best split, the algorithm only considers a random, limited subset.
- **Typical Defaults:** A common rule of thumb for the number of features to sample at each split is the square root of the total number of features for classification tasks, and one-third of the total features for regression tasks.
- **Purpose:** This process, also known as "feature bagging" or the "random subspace method", ensures that trees are not overly reliant on one or a few dominant features and encourages diversity among the individual trees, further reducing correlation and variance.

Sampling Techniques in Random Forest

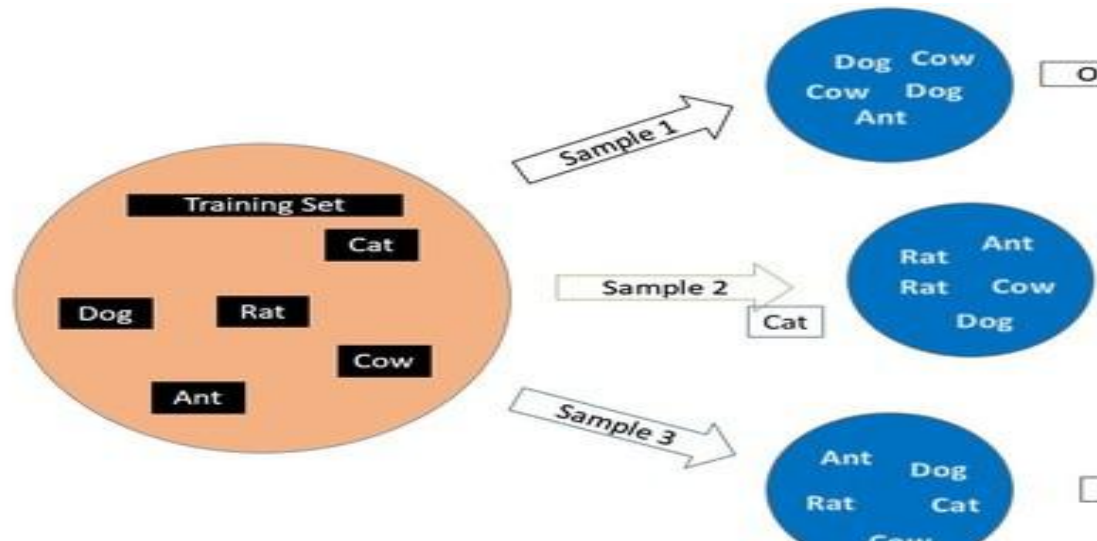
Summary of the Process

By combining these two sampling techniques, random forest builds a collection of diverse, uncorrelated trees. The final prediction is determined by aggregating the results of all trees (majority vote for classification, averaging for regression), which generally leads to more accurate, stable, and robust results than a single decision tree.



Out-of-Bag Sample in Random Forest

- ✓ The Random Forest algorithm comes along with the concept of Out-of-Bag Score(OOB_Score).
- ✓ you can observe that we repeated some animals while making the sample, and some animals did not even occur once in the sample.
- ✓ Here, Sample1 does not have Rat and Cow whereas sample 3 had all the animals equal to the main training set.
- ✓ While making the samples, data points were chosen randomly and with replacement, and the data points which fail to be a part of that particular sample are known as **OUT-OF-BAG** points.



Out-of-Bag Score (OOB_Score) in Random Forest

- **OOB_Score is a very powerful Validation Technique used especially for the Random Forest algorithm for least Variance results.**
- Note: While using the **cross-validation technique**, every validation set has already been seen or used in training by a few decision trees and hence there is a leakage of data, therefore more variance.
- But, OOB_Score prevents leakage and gives a better model with low variance, so we use OOB_score for validating the model.

Out-of-Bag Score (OOB_Score) in Random Forest

- Here, we have a training set with 5 rows and a classification target variable of whether the animals are domestic/pet?

Complete Training Set

Animals	Size of Animal ?	Can we Pet them?
Dog	Medium	Yes
Rat	Small	No
Ant	Small	No
Cow	Big	Yes
Cat	Medium	Yes

Out-of-Bag Score (OOB_Score) in Random Forest

- In the random forest, we build multiple decision trees. Below, we show a bootstrapped sample for one particular decision tree, say DT_1.
- Here, Rat and Cat data have been left out. And since, Rat and Cat are OOB for DT_1, we would predict the values for Rat and Cat using DT_1. (Note: Data of Rat and Cat hasn't been seen by DT_1 while training the tree.)
- Just like DT_1, there would be many more decision trees where either rat or cat was left out or maybe both of them were left out.

Out-of-Bag Score (OOB_Score) in Random Forest

- Let's say that the 3rd, 7th, and 100th decision trees have 'Rat' as an OOB datapoint. This means that none of them saw the 'Rat' data before predicting the value for 'Rat'.
- So, we recorded all the predicted values for "Rat" from the trees DT_1, Dt_3, DT_7, and DT_100.
- And saw that aggregated/majority prediction is the same as the actual value for "Rat".

(To Note: None of the models had seen data before, and still predicted the values for a data point correctly)

Sample1 Containing Dog , dog , cow , cow and ant			
Animals	Size of Animal ?	Can we Pet them?	
Dog	Medium	Yes	Bootstrapped Sample
Cow	Big	Yes	
Ant	Small	No	
Rat	Small	No	Out-Of-Bag
Cat	Medium	Yes	

Out-of-Bag Score (OOB_Score) in Random Forest

- Similarly, every data point is passed for prediction to trees where it would be behaving as OOB and an aggregated prediction is recorded for each row.
- The **OOB_score** is computed as the number of correctly predicted rows from the out-of-bag sample.
- **OOB Error** is the number of wrongly classifying the OOB Sample.

Decision Tree Number	Predictions
1	No
3	No
7	Yes
100	No
Majority	NO

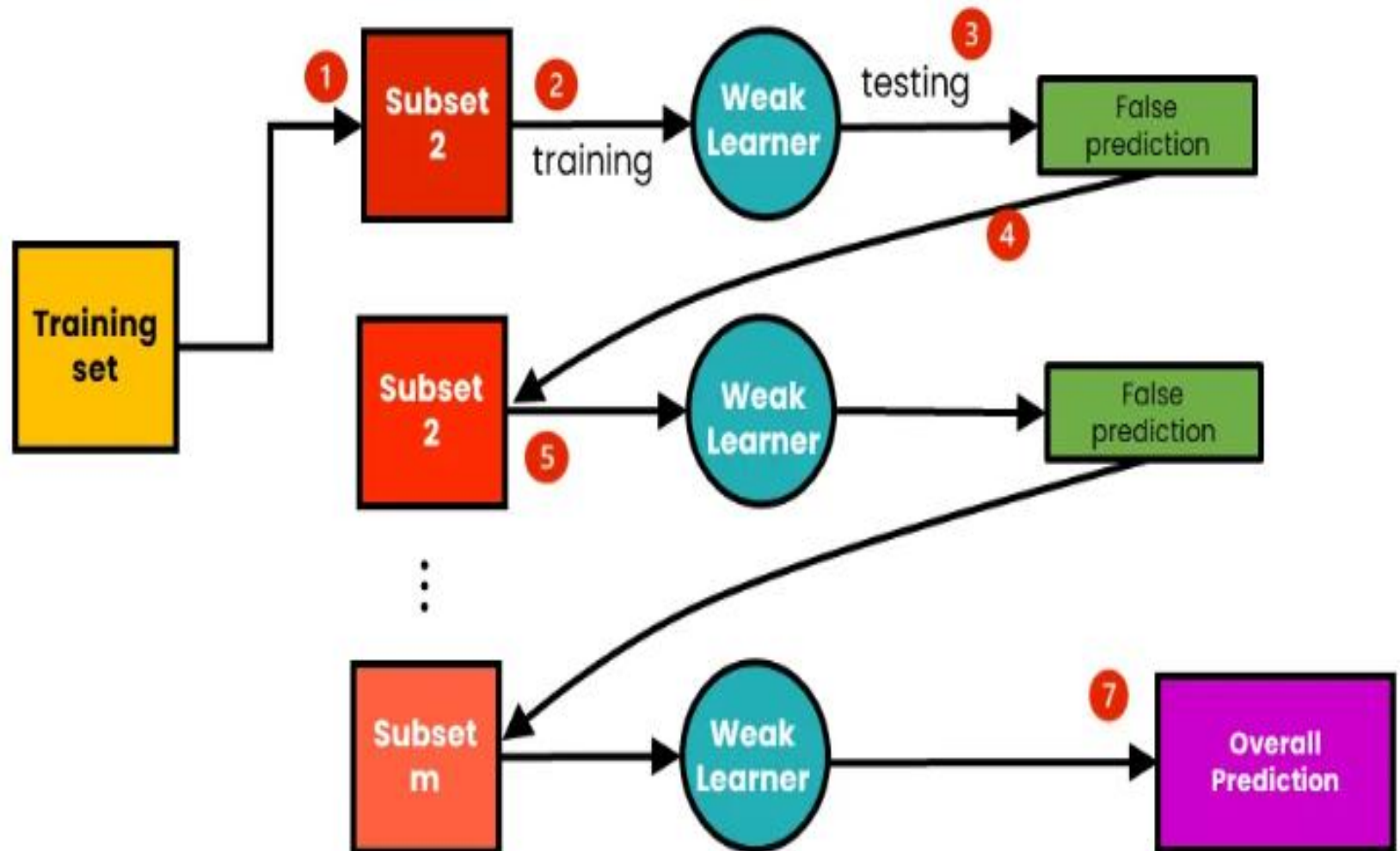
Out-of-Bag Score (OOB_Score) in Random Forest

- The Out-of-Bag score serves as a reliable validation score, offering an insight into the model's prediction error without the need for a separate validation dataset.
- By bypassing the necessity for a distinct validation dataset or test set, the Out-of-Bag error provides an efficient means to estimate the prediction error, streamlining the model evaluation process.
- The accuracy of the Out-of-Bag validation score highlights its effectiveness in reflecting the prediction error, making it a valuable tool for assessing model performance in the absence of an external validation dataset.

Boosting

- **How it works:** Models are trained **sequentially**. Each new model focuses on correcting the specific errors made by its predecessors by increasing the weights of misclassified data points.
- **Goal:** To reduce **bias** and improve overall accuracy.
- **Examples:** **AdaBoost**, **Gradient Boosting**, and high-performance libraries like **XGBoost**

The Process of Boosting



Bagging verses Boosting

- Bagging and Boosting are both ensemble learning techniques used to improve model performance by combining multiple models. The main difference is that:
- Bagging reduces variance by training models independently.
- Boosting reduces bias by training models sequentially, focusing on previous errors.

Bagging

- ✓ **Bagging:** aims to reduce model variance by training multiple models on different random subsets of the dataset. These subsets are created using bootstrapping, where data points are sampled with replacement.
- ✓ Each model is trained independently, and their predictions are combined using voting for classification or averaging for regression. As models are trained independently, Bagging works well with high-variance models like decision trees.
 - ✓ ■ Uses random sampling with replacement
 - ✓ ■ Trains models independently
 - ✓ ■ Combines predictions using voting or averaging
 - ✓ ■ Reduces overfitting
 - ✓ ■ Used in Random Forest, Credit risk prediction and Medical diagnosis systems

Boosting

- **Boosting** focuses on improving model accuracy by training models sequentially. Each new model pays more attention to the data points that were misclassified by previous models. Over time, the ensemble becomes better at handling difficult cases.
- Boosting is effective for reducing bias and works well even with weak learners.
- Trains models sequentially
- Gives more weight to misclassified samples
- Combines models using weighted voting
- Reduces bias
- Used in Fraud detection

Bagging verses Boosting



Feature	Bagging	Boosting
Training style	Trains independent models	Trains sequential models
Main goal	Reduce variance	Reduce bias
Handling errors	All samples treated equally	Focuses on misclassified samples
Overfitting	It is less sesitive to overfitting	More prone to overfitting
Parallel training	Yes it supports parallel computing	No it does not supports parallel computing
Sensitivity to noise	It is not affected by noise and outliers	Sensitive to noise and outliers

When to Use Bagging or Boosting

- Use Bagging when your model is overfitting and has high variance, especially with decision trees.
- Use Boosting when your model underfits and you want higher accuracy by learning complex patterns.

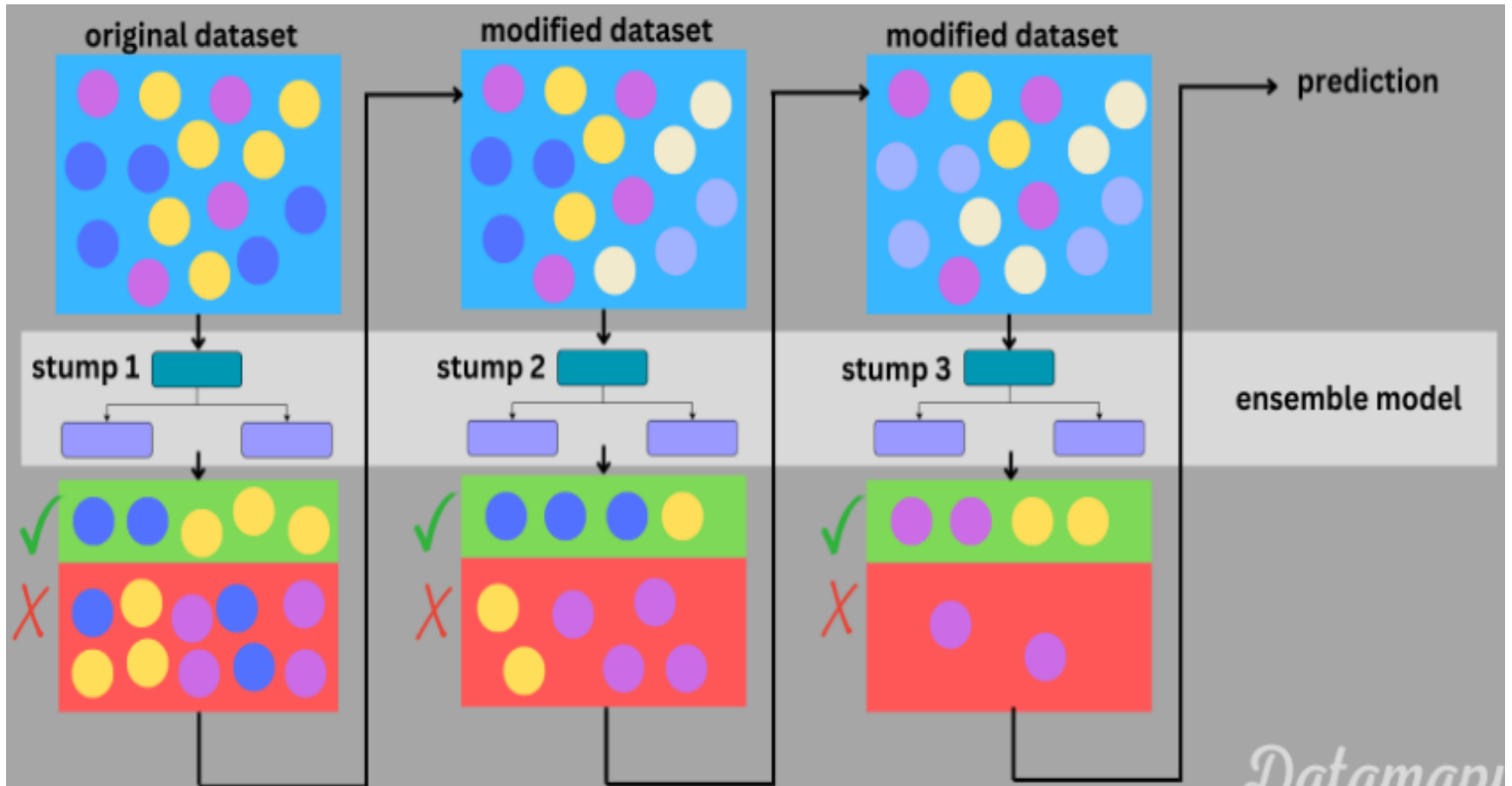
AdaBoost (Adaptive Boosting)

- **AdaBoost** (short for Adaptive Boosting) is a powerful ensemble learning technique that builds a "strong" classifier by **sequentially** combining many "weak" learners.
- It is widely used for classification tasks because it improves accuracy by focusing specifically on the data points that previous models struggled to predict correctly.
- In adaboost, we majorly used **decision stumps** as the **weak learners**.

AdaBoost

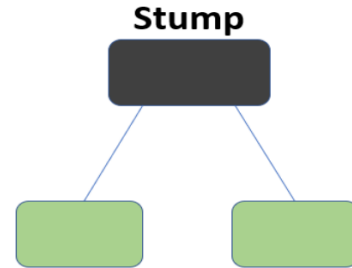
- ✓ AdaBoost is a Boosting algorithm, which means that the ensemble model is built sequentially and each new model builds on the results of the previous one, trying to improve its errors.
- ✓ The developed models are all *weak-learners*, that is they have low predictive skill which is only slightly higher than random guessing.
- ✓ The word *adaptive* refers to the adaption of the weights that are assigned to each sample before fitting the next model.
- ✓ The weights are determined in such a way that the wrongly predicted samples get higher weights than the correctly predicted samples.

AdaBoost



AdaBoost

- The most common algorithm used with AdaBoost is **decision trees with one level** that means with Decision trees with only 1 split. These trees are also called **Decision Stumps**



- What this algorithm does is that it builds a model and gives equal weights to all the data points. It then assigns higher weights to points that are wrongly classified.
- Now all the points which have higher weights are given more importance in the next model. It will keep training models until and unless a lower error is received.

AdaBoost Algorithm

- The above diagram explains the AdaBoost algorithm in a very simple way. Let's try to understand it in a stepwise process:
- **Step 1: Initial Model (B1)**
- The dataset consists of multiple data points (red, blue and green circles).
- Equal weight is assigned to each data point.
- The first weak classifier attempts to create a decision boundary.

AdaBoost Algorithm

- **Step 2: Adjusting Weights (B2)**

- The misclassified points from B1 are assigned higher weights (shown as darker points in the next step).
- A new classifier is trained with a refined decision boundary focusing more on the previously misclassified points.
- Some previously misclassified points may now be correctly classified.

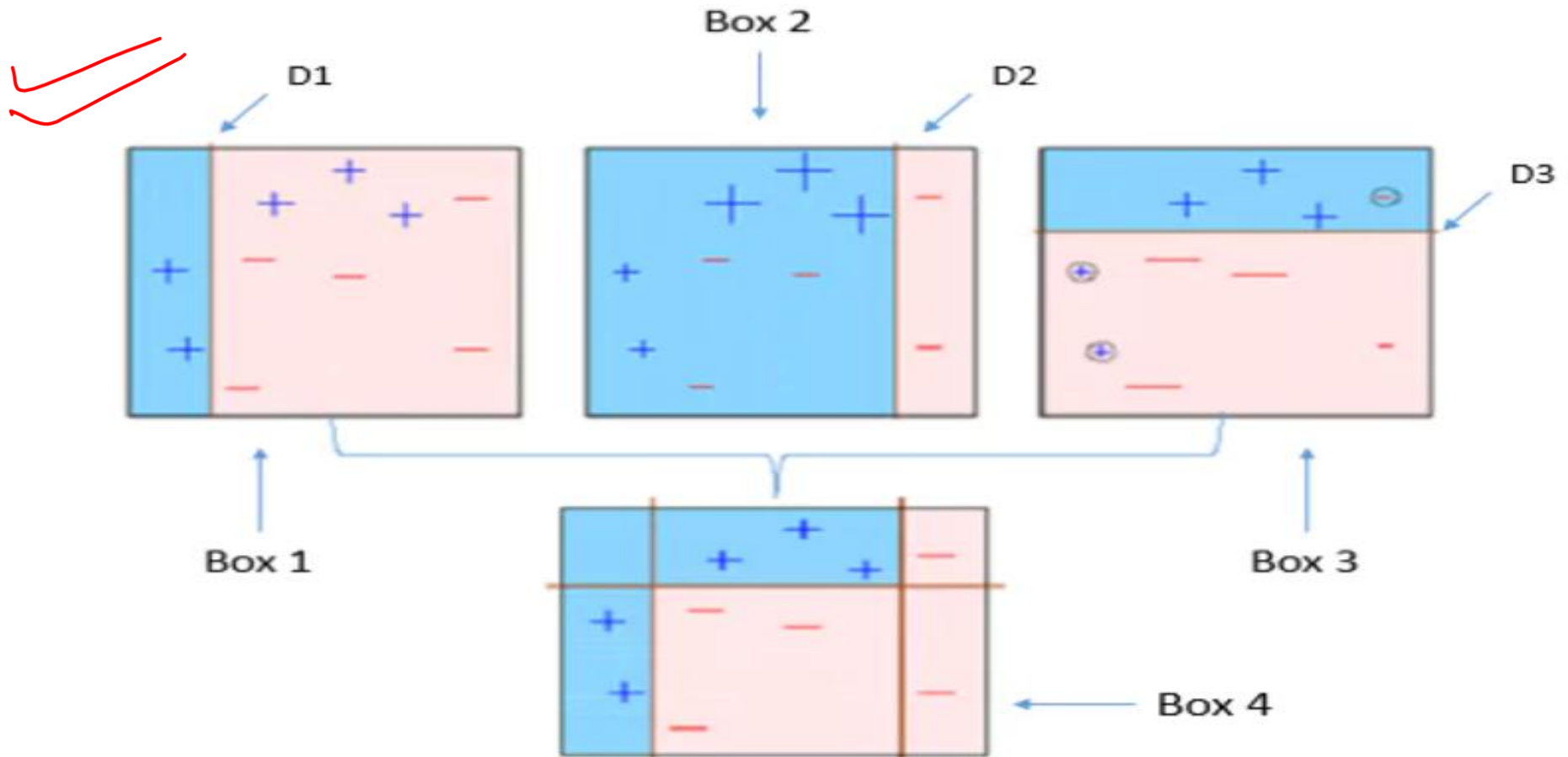
- **Step 3: Further Adjustment (B3)**

- The newly misclassified points from B2 receive higher weights to ensure better classification.
- The classifier adjusts again using an improved decision boundary and some data points remain misclassified.

AdaBoost Algorithm

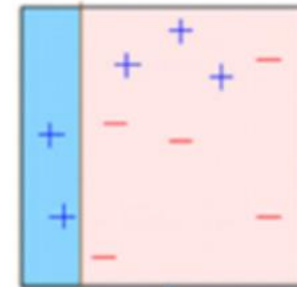
- **Step 4: Final Strong Model (B4 - Ensemble Model)**
- The final ensemble classifier combines B1, B2 and B3 to get strengths of all weak classifiers.
- By aggregating multiple models the ensemble model achieves higher accuracy than any individual weak model.

Prediction in AdaBoost



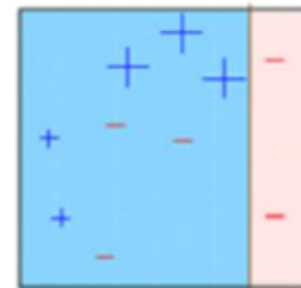
Prediction in AdaBoost

- **Box 1:** You can see that we have assigned equal weights to each data point and applied a decision stump to classify them as + (plus) or - (minus). The decision stump (D1) has generated vertical line at left side to classify the data points. We see that, this vertical line has incorrectly predicted three + (plus) as - (minus). In such case, we'll assign higher weights to these three + (plus) and apply another decision stump.



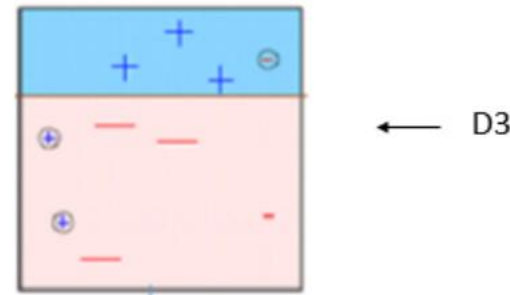
Prediction in AdaBoost

- **Box 2:** Here, you can see that the size of three incorrectly predicted + (plus) is bigger as compared to rest of the data points. In this case, the second decision stump (D2) will try to predict them correctly. Now, a vertical line (D2) at right side of this box has classified three mis-classified + (plus) correctly. But again, it has caused mis-classification errors. This time with three -(minus). Again, we will assign higher weight to three – (minus) and apply another decision stump.



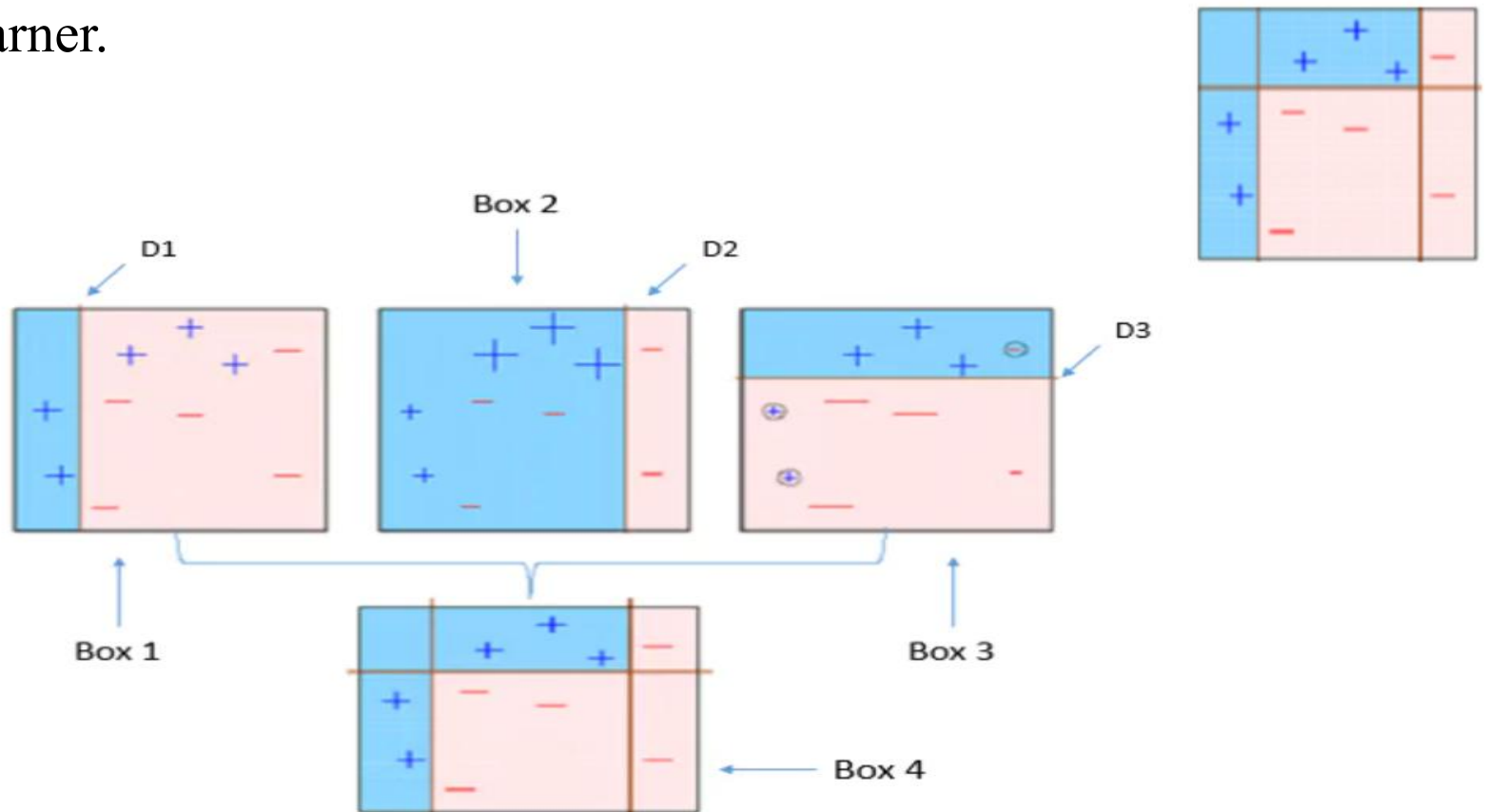
AdaBoost Algorithm

- **Box 3:** Here, three $-$ (minus) are given higher weights. A decision stump (D3) is applied to predict these mis-classified observation correctly. This time a horizontal line is generated to classify $+$ (plus) and $-$ (minus) based on higher weight of mis-classified observation.

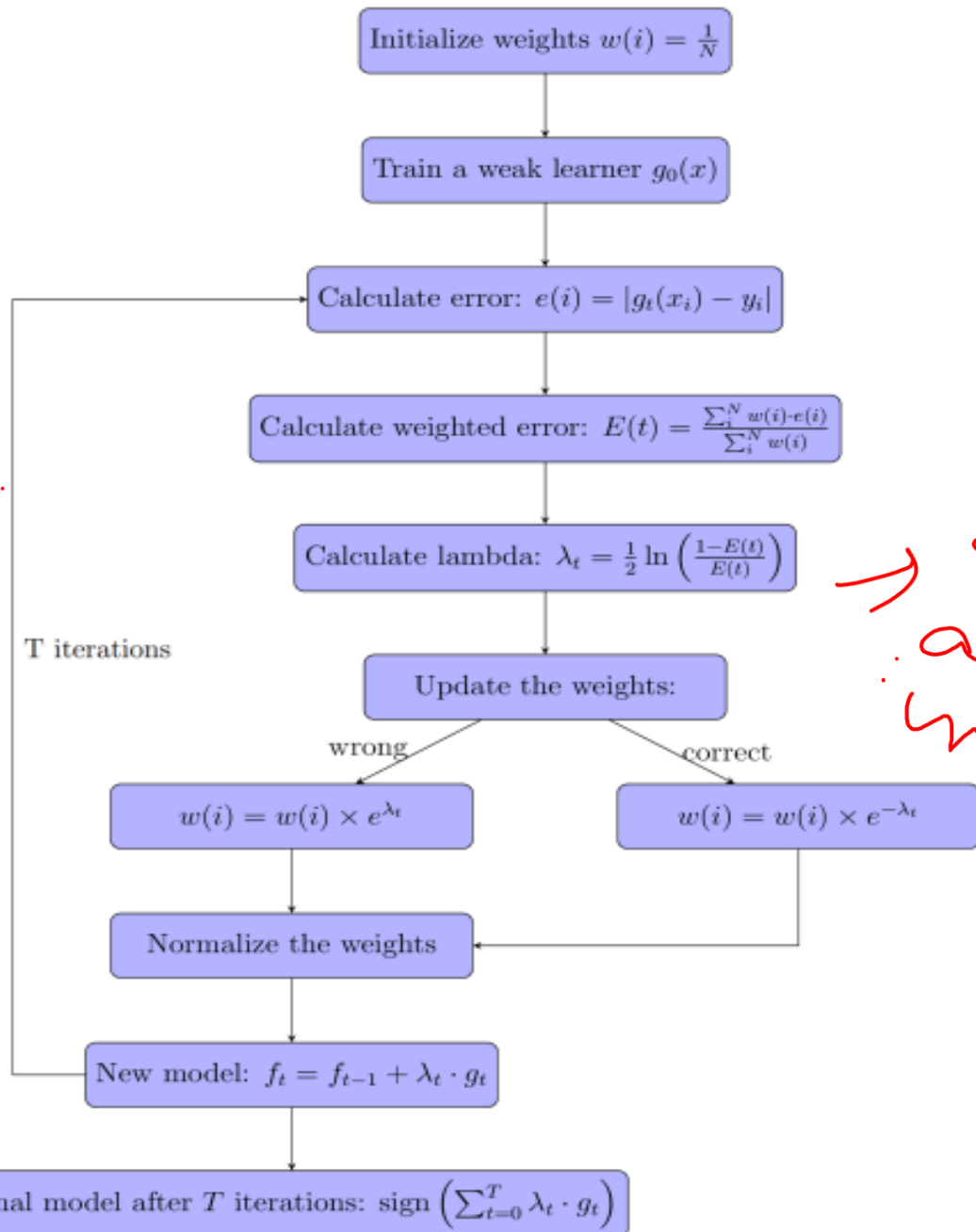


AdaBoost Algorithm

- **Box 4:** Here, we have combined D1, D2 and D3 to form a strong prediction having complex rule as compared to individual weak learner. You can see that this algorithm has classified these observation quite well as compared to any of individual weak learner.



The Algorithm



AdaBoost

λ_t are same in AdaBoost

Step 1 of AdaBoost Algorithm

- **1. Fit a model to the initial dataset with equal weights:** The first step is to assign a weight to each sample of the dataset.
- The **initial weight** is $1/n$, with n being the number of data points. The weights always sum up to 1. Because in the beginning, all weights are equal, this means they can be ignored in this first step.
- If the base model is a Decision Tree, the weak learner is a very shallow tree or even only the *stump*, which refers to the tree that consists only of the root node and the first two leaves.
- How deep the tree is developed is a hyperparameter, that needs to be set. If we fit an AdaBoost algorithm in Python and use sklearn the default setting depends on whether a classification or a regression is considered.
- For classification, the default setting is 1 to use stumps and for regression, the default value is a maximal depth of 3.

Step 2 of AdaBoost Algorithm

- **2. Make predictions and calculate the *influence* (α) of the fitted model.** The weights will be assigned to dataset as well as to each model.
- The weights that are associated with each model their *influence*.
- The influence of a model on the final prediction depends on its error and is calculated as

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - TotalError}{TotalError} \right)$$

- The *Total Error* is the sum of the sample weights for all wrongly predicted data points, which is always between 0 and 1.
- With 0 meaning the model predicts all samples wrongly and 1 meaning all samples are correctly predicted.

Step 2 of AdaBoost Algorithm: Error Rate versus alpha

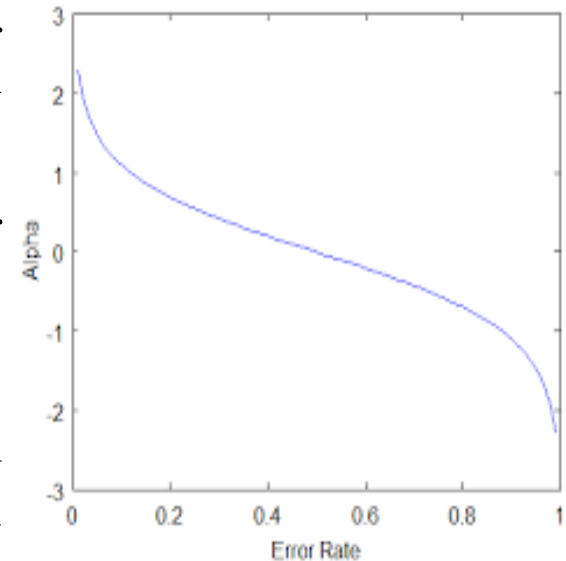
- **Influence:** In AdaBoost, "influence" refers to the weight (α) assigned to each weak learner (typically a decision stump), determining how much say it has in the final ensemble prediction, with more accurate models receiving higher influence.

- **Key Thresholds**

- **Error < 0.5:** The classifier is better than random guessing. The term inside the natural log is greater than 1, resulting in a positive alpha.

- **Error = 0.5:** The classifier is no better than a coin flip. The term inside the log is 1, resulting in an alpha of 0.

- **Error > 0.5:** The classifier is worse than random. This results in a negative alpha, meaning the algorithm will actually flip the classifier's prediction to improve the ensemble.



Step 3 of AdaBoost Algorithm

- **Step 3. Adjust the weights of the data samples.** The new weight (w_{new}) for each data sample is calculated as

$$w_{\text{new}} = w_{\text{old}} * e^{\pm\alpha},$$

- with the old or previous weight of that sample and the influence of the model calculated in the previous step.
- The sign in the exponent changes depending on whether the sample was correctly predicted or not.
- If it was correctly predicted, the sign is negative, so that the weight decreases.
- On the other hand, if it was wrongly predicted, the sign is positive, so that the weight increases.
- Because the sum of all weights must be 1, they are normalized, by dividing them by their sum.

Step 4 of AdaBoost Algorithm

- **Step 4: Create a weighted dataset.** The calculated weights are now used to create a new dataset. For that, the calculated weights are used as bins for each sample.
- Let's assume we calculated the weights $w_1, w_2, w_3, \dots, w_n$, the bin for the first sample is 0 to w_1 .
- The bin for the second sample is w_1 to $w_1 + w_2$ and so on. Data samples are now selected by choosing random numbers between 0 and 1, with N the number of data samples.
- For each of these random numbers, the sample is chosen in which bin the random number falls. Since wrongly predicted samples have higher weight than correctly predicted samples, their bins are larger. Therefore the probability of drawing a wrongly predicted sample is higher. The newly created dataset consists again of samples, but there are likely duplicates from the wrongly predicted samples.

Step 5 and iteration of AdaBoost Algorithm

- **Step 5: Fit a model to the new dataset.** Now we start again and fit a model, equally to the first step, but this time using the modified dataset.
- Repeat steps 2 to 5 d times, where d is the number of final weak learners of which the ensemble model is composed. It is a hyperparameter that needs to be chosen.

Final Prediction in AdaBoost Algorithm

- For the final prediction of the ensemble model, predictions of all individual models are made.
- In a classification task, the influences of the different predictions are added and the majority is the final prediction.
- In a regression task, the weighted mean of the individual model prediction gives the final prediction, where the weights are the influences of the models.